

Jenkins

To Begin with the Lab

Starting with Jenkins.

Summary of the Lab

In this lab, a Linux EC2 instance is created via the AWS Console. After launching the instance, connection is made using EC2 Instance Connect. Jenkins is installed by running a series of commands including installing Java, configuring the Jenkins repository, and enabling the Jenkins service. The Jenkins web interface is accessed via port 8080 using the public DNS. If inaccessible, inbound rules are modified in the security group. The Jenkins initial password is retrieved using a terminal command, and the setup is completed by installing suggested plugins and creating the first admin user, enabling full access to Jenkins dashboard.

- Open the Amazon EC2 console at [AMAZON EC2](#).
- Choose Launch instance.
- (Optional) Under Name and tags, for Name, enter a name to identify your instance.
- Under Application and OS Images (Amazon Machine Image), choose a Linux operating system, and then for Amazon Machine Image (AMI), select a Linux AMI.

Name and tags Info

Name [Add additional tags](#)

Application and OS Images (Amazon Machine Image) Info

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Search our full catalog including 1000s of application and OS images

Recents | My AMIs | **Quick Start**

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type Free tier eligible

ami-020c3a7c55df1f615 (64-bit (x86)) / ami-07041441b708acbd6 (64-bit (ARM))

Virtualization: hvm | ENA enabled: true | Root device type: ebs

Description

Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

Architecture **AMI ID** **Publish Date** **Username**

64-bit (x86) | ami-020c3a7c55df1f615 | 2025-06-10 | ubuntu Verified provider

Summary

Number of instances

Software image (AMI)
Canonical, Ubuntu, 24.04, amd64... [read more](#)
ami-020c3a7c55df1f615

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GB of bandwidth to the internet.

[Cancel](#) [Launch instance](#) [Preview code](#)

- Under Instance type, for Instance type, select an instance type or keep the default.

Instance type Info | Get advice

Instance type

t2.micro Free tier eligible

Family: t2 | 1 vCPU | 1 GiB Memory | Current generation: true | On-Demand Windows base pricing: 0.0162 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0134 USD per Hour | On-Demand SUSE base pricing: 0.0116 USD per Hour

On-Demand RHEL base pricing: 0.026 USD per Hour | On-Demand Linux base pricing: 0.0116 USD per Hour

[Additional costs apply for AMIs with pre-installed software](#)

☐ All generations [Compare instance types](#)

- Use 16 GiB.

- ▼ Configure storage info

1 x GiB Root volume, 3000 IOPS, Not encrypted

① Free tier eligible customers can get up to 30 GiB of EBS General Purpose (SSD) or Magnetic storage

Add new volume

② Click refresh to view backup information

The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

0 x File systems

AdvancedEdit

Storage (volumes)

1 volume(s) - 8 GiB

① Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet.

Cancel

Launch instance

Preview code

- Instances** (1/2) [Info](#)

Find Instance by attribute or tag (case-sensitive)

[Connect](#) [Instance state ▾](#) [Actions ▾](#) [Launch instances ▾](#)

	Name ↗	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status	Availability Zone ▾	Public IPv4 DNS ▾	Public
☒	Jenkins	i-0e7f4b39c5be66fd7	● Running 🔍	t2.micro	● 2/2 checks passsec View alarms +		us-east-1b	ec2-54-236-136-85.co...	54.236

- EC2

Instances

i-0e7f4b39c5be66fd7

Connect to instance

Connect

info

Connect to an instance using the browser-based client.

EC2 Instance Connect

Session Manager

SSH client

EC2 serial console

Instance ID

i-0e7f4b39c5be66fd7 (Jenkins)

Connect using a Public IP

Connect using a public IPv4 or IPv6 address

Connect using a Private IP

Connect using a private IP address and a VPC endpoint

Public IPv4 address

54.236.136.85

IPv6 address

—

Username

Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.

Q ec2-user

X

Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel

Connect

- ```

#
####
#####\
#####|
##/
V-+ -> https://aws.amazon.com/linux/amazon-linux-2023
~
~
~/m/ +
[ec2-user@ip-172-31-84-161 ~]$
```

- ```
[ec2-user@ip-172-31-84-161 ~]$ sudo yum update -y
Last metadata expiration check: 0:08:01 ago on Sun Jul  6 10:46:11 2025.
No match for argument: -y
Error: No packages marked for upgrade.
[ec2-user@ip-172-31-84-161 ~]$
```

- So, basically run the given commands one after the other

- `sudo wget -O /etc/yum.repos.d/jenkins.repo \`
- `https://pkg.jenkins.io/redhat-stable/jenkins.repo`
- `sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key`
- `sudo yum upgrade`
- `sudo yum install java-21-amazon-corretto -y`
- `sudo yum install jenkins -y`
- `sudo systemctl enable jenkins`
- `sudo systemctl start jenkins`
- `sudo systemctl status jenkins`
- After running the ‘`sudo systemctl status jenkins`’ command, check the status.
- The status should Active (running)

```
[ec2-user@ip-172-31-84-161 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Sun 2025-07-06 10:59:36 UTC; 1min 44s ago
     Main PID: 27398 (java)
        Tasks: 38 (limit: 1111)
      Memory: 365.5M
         CPU: 17.666s
    CGroup: /system.slice/jenkins.service
            └─27398 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080
```

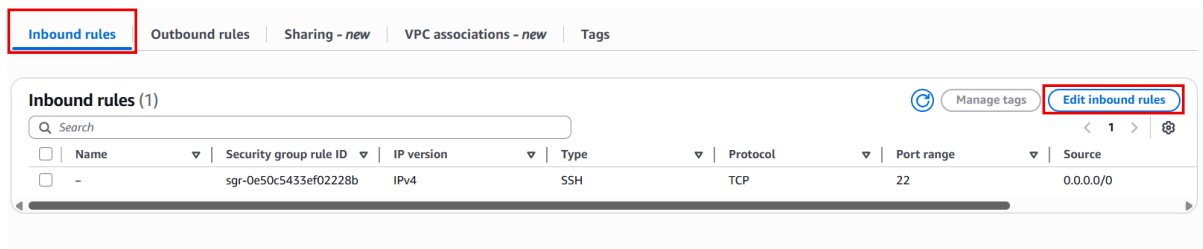
- Now Jenkins is installed.
- Go back to your instance.
- Copy the public DNS

The screenshot shows the 'Instance summary' page for an AWS EC2 instance named 'i-0e7f4b39c5be66fd7 (Jenkins)'. The 'Public DNS' field is highlighted with a red box, displaying 'ec2-54-2...aws.com'. Other visible fields include 'Instance ID', 'IPv6 address', 'Hostname type', 'Public IPv4 address', 'Private IPv4 addresses', 'Instance state' (Running), and 'Private IP DNS name'.

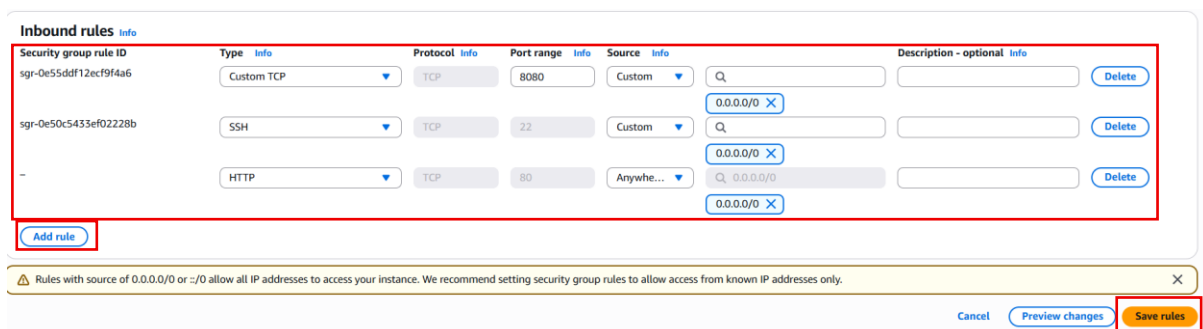
- Now connect to your browser.
- `http://<your_server_public_DNS>:8080`
- if it doesn't open.
- And the error message is ‘This site can't be reached’
- Go to instance.
- Go to security and click on security groups link.

The screenshot shows the 'Security details' page for the same AWS EC2 instance. The 'Security groups' field is highlighted with a red box, displaying 'sg-01ddbc0b881e917a5 (launch-wizard-2)'. Other visible fields include 'IAM Role', 'Owner ID', and 'Launch time'.

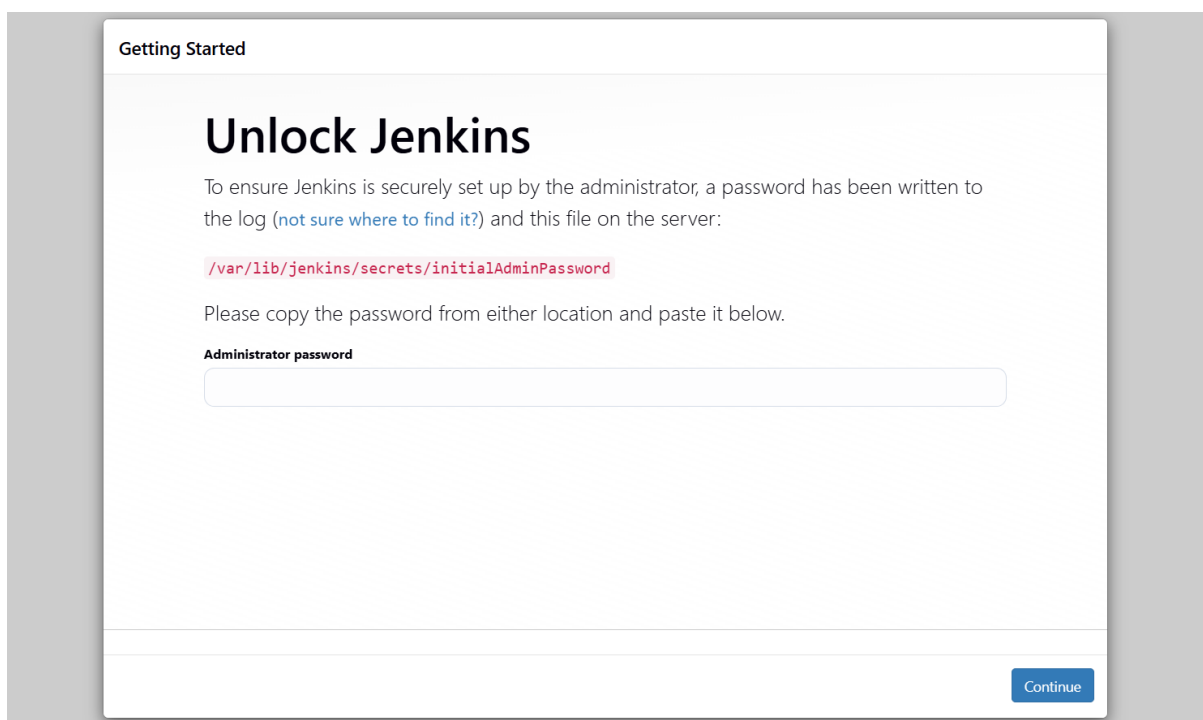
- Click on the inbound rule.
- Click on the ‘edit inbound rules’



- Click on the Add rule.
- Make the changes accordingly.
- And save the rules.



- After that you can access the jenkins page.



- As prompted, enter the password found in `/var/lib/jenkins/secrets/initialAdminPassword`.
- For that run the command - `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
- Copy the password and paste it.
- And click continue.

```
[ec2-user@ip-172-31-84-161 ~]$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
6bdded2ee6[REDACTED]
[ec2-user@ip-172-31-84-161 ~]$
```

- The Jenkins installation script directs you to the **Customize Jenkins** page. Click **Install suggested plugins**.
- Once the installation is complete, the **Create First Admin User** will open. Enter your information, and then select **Save and Continue**.

Getting Started

Create First Admin User

Username:	admin
Password:	
Confirm password:	
Full name:	
E-mail address:	

Jenkins 2.263.1

[Skip and continue as admin](#)

[Save and Continue](#)

- and then, Click save and finish.
- And Start Using Jenkins.

Creating a Jenkins Job

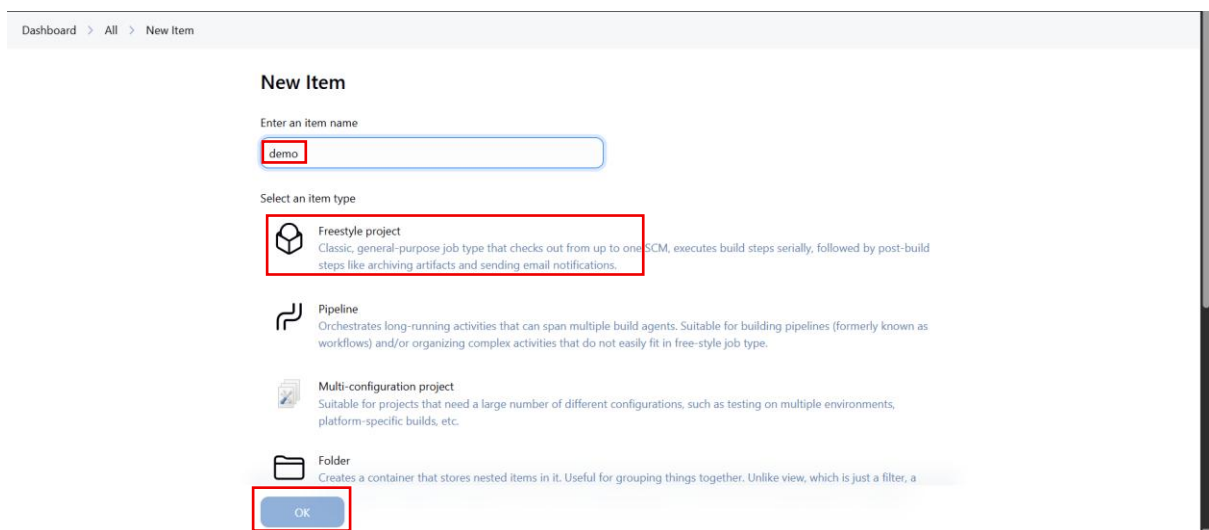
Summary of the Lab

Once Jenkins is set up, a Freestyle Project job is created. After naming the job and selecting the project type, a simple shell command (echo "Hello World") is added in the Build Step. Upon saving the configuration, the job is executed using the **Build Now** option. Each build generates a unique ID (e.g., #1, #2) where outputs can be viewed via Console Output. Status such as “SUCCESS” confirms successful execution. Job configurations can be edited by selecting **Configure**. This section demonstrates the basics of job creation, execution, result viewing, and reconfiguration within the Jenkins interface.

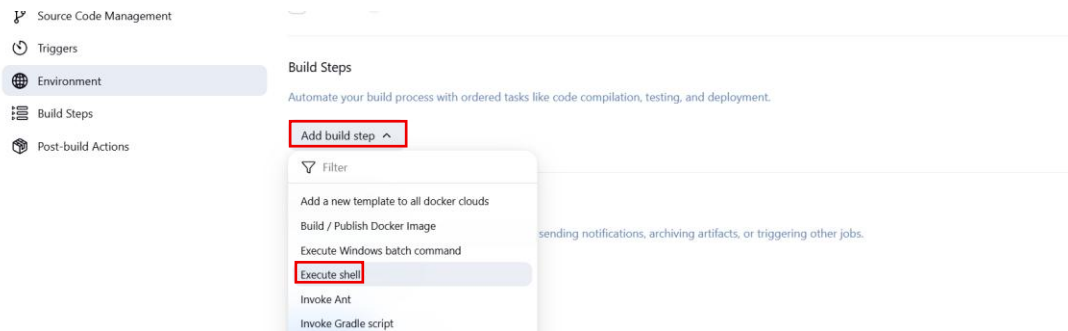
- Login to the Jenkins console
- Click on New Items.



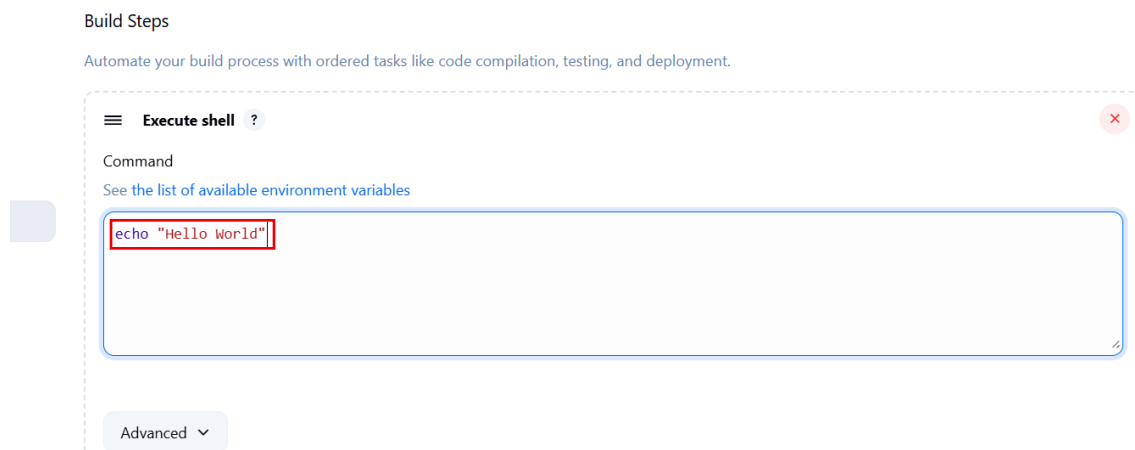
- Give the name of the job
- Then select the freestyle project
- And click ok.



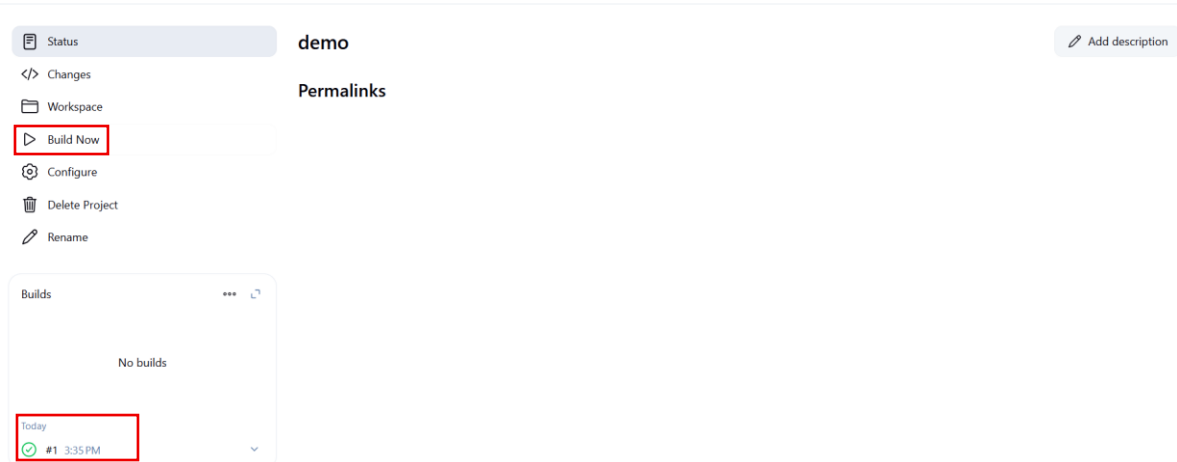
- Navigate to the Build Step section
- Click on the Add Build Step
- Select the 'Excute Shell'



- A shell will open.
- Type a simple command – echo “Hello World”



- Click Save.
- After saving it will create a job successfully
- Now click on the Build now
- It will excute your job.
- When you will click on the ‘Build Now’ it sends a notification ‘build schedule’
- You can click on the #1 to see the results under build section.



- Then you will be navigated to #1 build page.
- Then click on the Console Output.

This screenshot shows the Jenkins build #1 console output page. The left sidebar contains a menu with 'Status' (selected), 'Changes', 'Console Output' (highlighted with a red box), 'Edit Build Information', 'Delete build '#1'', and 'Timings'. The main content area shows a green checkmark icon and the text '#1 (Jul 10, 2025, 3:35:05 PM)'. Below this, it says 'Started by user Aman Rathor' and 'This run spent: 51 ms waiting; 97 ms build duration; 0.14 sec total from scheduled to completion.' At the bottom, it says '</> No changes.' On the right, there are buttons for 'Add description' and 'Keep this build forever', and a status message: 'Started 4 min 1 sec ago Took 97 ms'.

- You can see the status here.
- Here you get the SUCCESS message after your job finish successfully.
- Here you can delete your builds.

This screenshot shows the Jenkins build #1 console output page. The left sidebar contains a menu with 'Status', 'Changes', 'Console Output' (selected), 'Edit Build Information', 'Delete build '#1'', and 'Timings'. The main content area shows a green checkmark icon and the text 'Console Output'. Below this, there is a large text area containing the output of the build: 'Started by user Aman Rathor', 'Running as SYSTEM', 'Building in workspace /var/lib/jenkins/workspace/demo', '[demo] \$ /bin/sh -xe /tmp/jenkins3361042612828515201.sh', '+ echo 'Hello World'', 'Hello World', and 'Finished: SUCCESS'. On the right, there are buttons for 'Download', 'Copy', and 'View as plain text'.

- Now click on the demo

This screenshot shows the Jenkins demo job page. The breadcrumb navigation at the top reads 'Dashboard > demo > #1 > Console Output'. The left sidebar contains a menu with 'Status', 'Changes', 'Workspace', 'Build Now', 'Configure' (highlighted with a red box), 'Delete Project', and 'Rename'. The main content area shows a green checkmark icon and the text 'demo'. Below this, there is a section titled 'Permalinks' with a list of links: 'Last build (#1), 9 min 43 sec ago', 'Last stable build (#1), 9 min 43 sec ago', 'Last successful build (#1), 9 min 43 sec ago', and 'Last completed build (#1), 9 min 43 sec ago'.

- Here I have made changes in the Shell.
- Click Save

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps**
- Post-build Actions

☐ Terminate a build if it's stuck

☐ Terraform

☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

[See the list of available environment variables](#)

```
echo "Happy Learnings"
```

Advanced ▾

Build Step ▾

Save Apply

- After saving, Click on Build Now.
- Your #2 build will create.

Builds

⋮

🔍 Filter

/

Today

✓ #2 3:52 PM

✓ #1 3:35 PM

- You can click on it and can check the console output.

Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Timings

Previous Build

✓ **Console Output**

Download

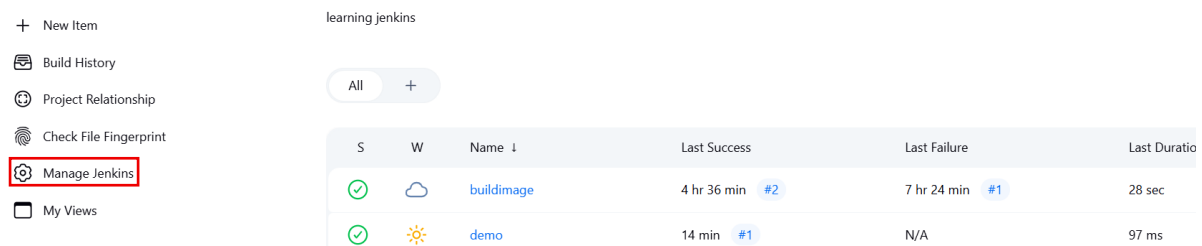
```
Started by user Aman Rathor
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/demo
[demo] $ /bin/sh -xe /tmp/jenkins2209228576631348294.sh
+ echo 'Happy Learnings'
Happy Learnings
Finished: SUCCESS
```

Saving Credential in Jenkins

Summary of the Lab

To securely connect Jenkins with services like GitHub, credentials are saved in Jenkins. From the **Manage Jenkins** menu, users can access **Credentials**, navigate to **global scope**, and add new credentials. For GitHub, a personal access token is generated under Developer Settings > Personal Access Tokens. The token, once generated, is copied and saved as the password in Jenkins along with the GitHub username. This stored credential can then be reused for Git-based project jobs. It ensures secure and streamlined integration with GitHub without exposing plain-text passwords or repeatedly generating tokens for every Jenkins project.

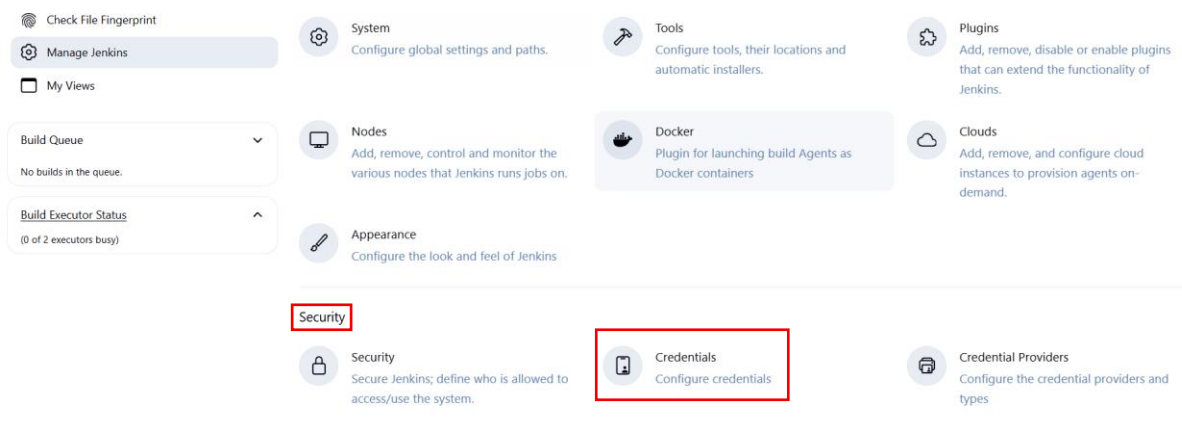
- Go to Dashboard.
- Click on Manage Jenkins.



The screenshot shows the Jenkins dashboard for a workspace named 'learning jenkins'. On the left sidebar, the 'Manage Jenkins' option is highlighted with a red box. The main area displays a table of build jobs.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	buildimage	4 hr 36 min #2	7 hr 24 min #1	28 sec
✓	☀	demo	14 min #1	N/A	97 ms

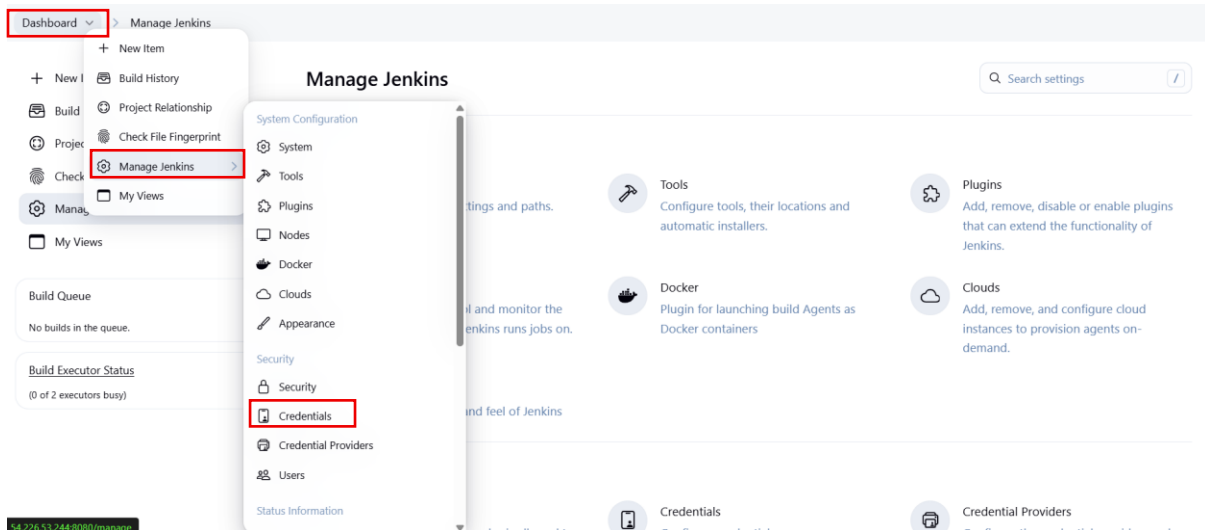
- Click on the Credentials under security.



The screenshot shows the 'Manage Jenkins' page. The 'Security' section is expanded, and the 'Credentials' option is highlighted with a red box. The page includes various configuration options like System, Tools, Plugins, Nodes, Docker, Clouds, and Appearance.

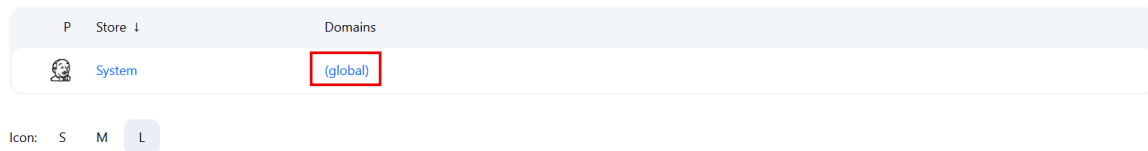
OR

- You can directly visit to Credentials through a dropdown list.
- Place your cursor on the Dashboard
- Then place your cursor on the Manage Jenkins
- Then click on the Credentials.



- In 'Stores scoped to Jenkins'
- Click on global.
- Then Click on the Add credentials

Stores scoped to Jenkins

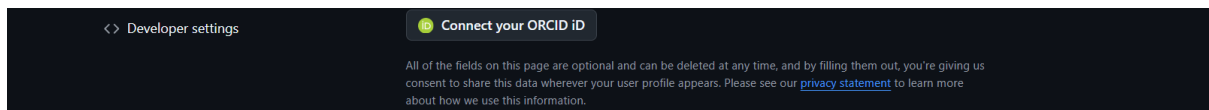


- Now you will be redirected to New Credential page

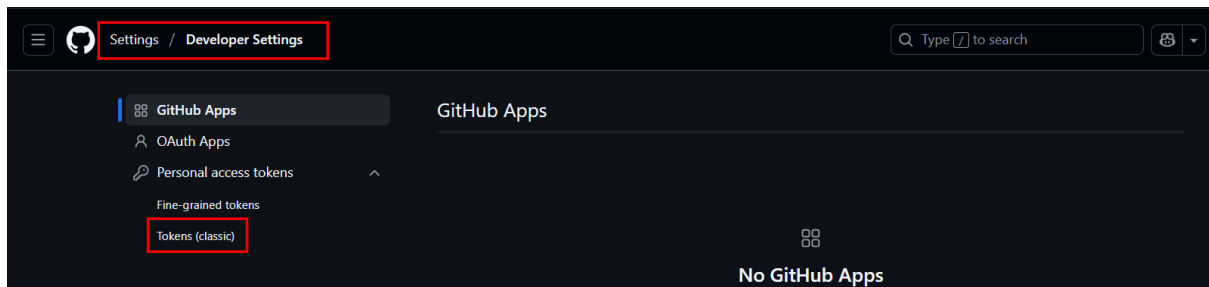
A screenshot of the 'New credentials' form. The form has a 'Kind' dropdown menu set to 'Username with password'. Below it is a 'Scope' dropdown menu set to 'Global (jenkins, nodes, items, all child items, etc)'. There are input fields for 'Username' and 'Password'. A checkbox labeled 'Treat username as secret' is present. At the bottom, there is a 'Create' button.

- Now here I will use my GitHub credential for that.
- Add your user name
- Now for password we will use token from the GitHub for that;
- Go to your GitHub account.
- Go to your GitHub profile.

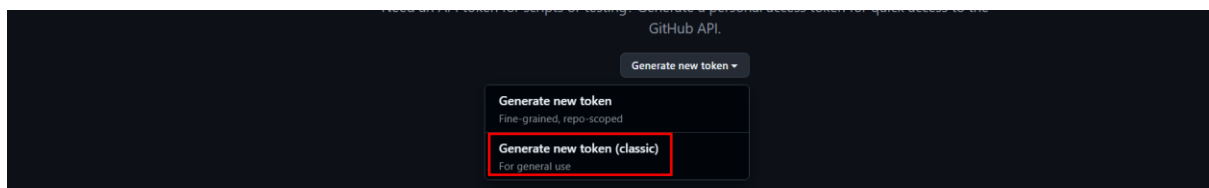
- Go to settings
- Navigate to Developer settings and click on it.



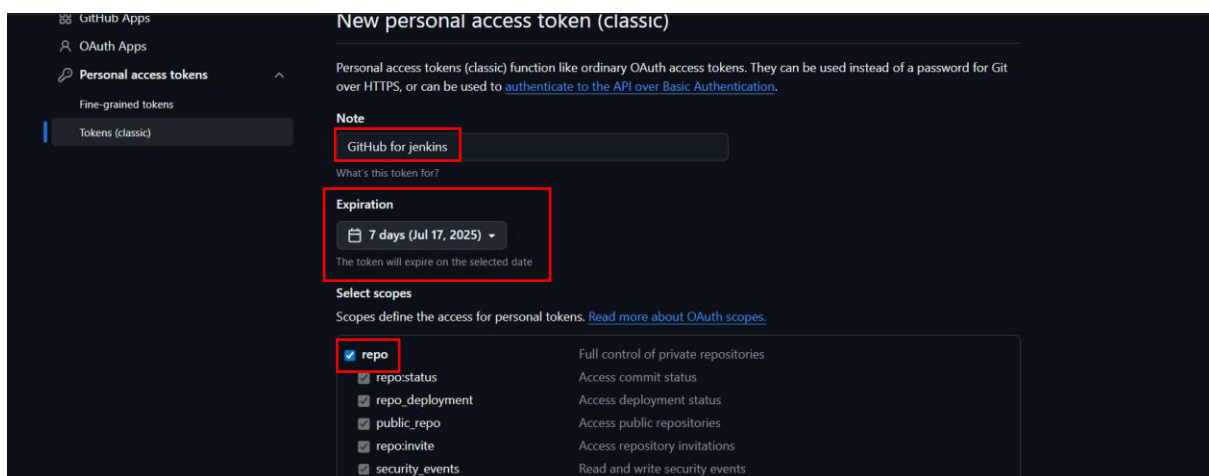
- Click on personal access token.
- Click on the Token(classic).



- Click on the Generate new token.
- Choose the Generate new token (classic).

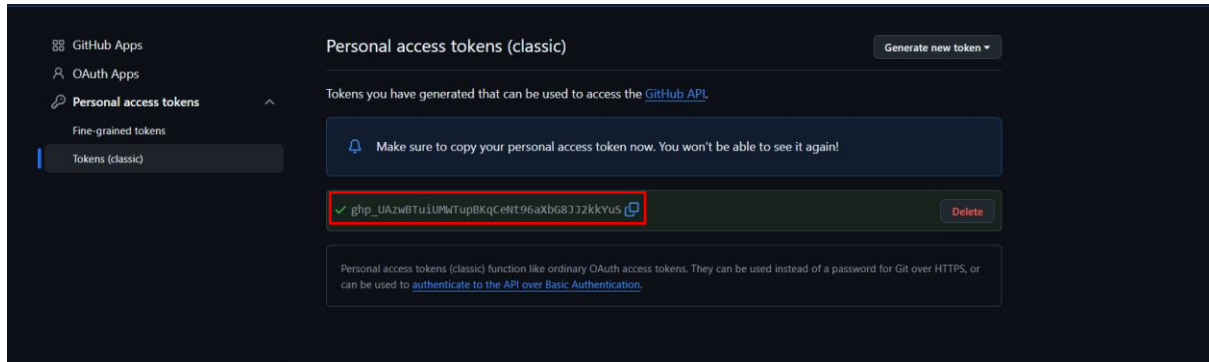


- Select and add the details given below
- Give the Note
- Select the days for expiry of the token.
- From 'select scope'
- Select repo, admin:repo_hook, notifications.
- Click Generate Token.



☒ admin:repo_hook	Full control of repository hooks
☒ write:repo_hook	Write repository hooks
☒ read:repo_hook	Read repository hooks
☐ admin:org_hook	Full control of organization hooks
☐ gist	Create gists
☒ notifications	Access notifications

- Your token will be generated



- Copy the token and paste in the password

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

ar4596472

☐ Treat username as secret ?

Password ?

.....

ID ?

github

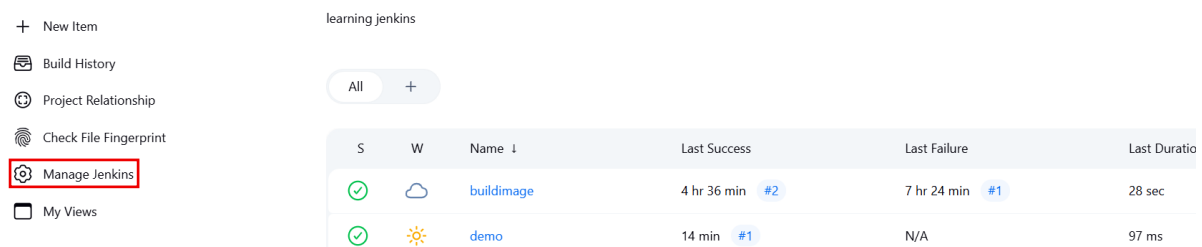
- And Click create.
- And you can add the credentials for the other things like docker
- Note: you can generate token for few account and use it not for every.

Jenkins Plugin

Summary of the Lab

Jenkins plugins extend core functionalities. From the **Manage Jenkins** page, users can view **Installed Plugins** or access the **Available Plugins** tab to add new ones. The GitHub plugin is commonly pre-installed. To support container workflows, the Docker plugin is searched, selected, and installed directly from the plugin manager. Once downloaded, Jenkins is enhanced with Docker integration capabilities. Proper plugin management ensures Jenkins is equipped to handle various stages of CI/CD pipelines, code repositories, cloud integrations, and more. Staying updated with the required plugins is essential for maintaining a flexible and powerful Jenkins environment.

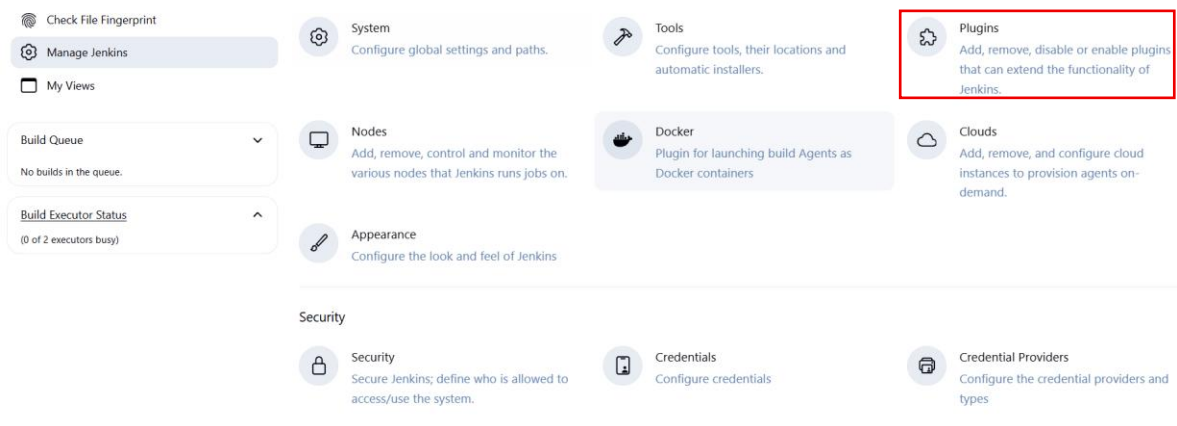
- Go to Dashboard.
- Click on Manage Jenkins.



The screenshot shows the Jenkins 'Manage Jenkins' page. On the left sidebar, 'Manage Jenkins' is highlighted with a red box. The main content area displays a table of installed plugins. The table has columns for Status (S), Icon (W), Name, Last Success, Last Failure, and Last Duration. Two plugins are listed: 'buildimage' and 'demo'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	buildimage	4 hr 36 min #2	7 hr 24 min #1	28 sec
✓	☀	demo	14 min #1	N/A	97 ms

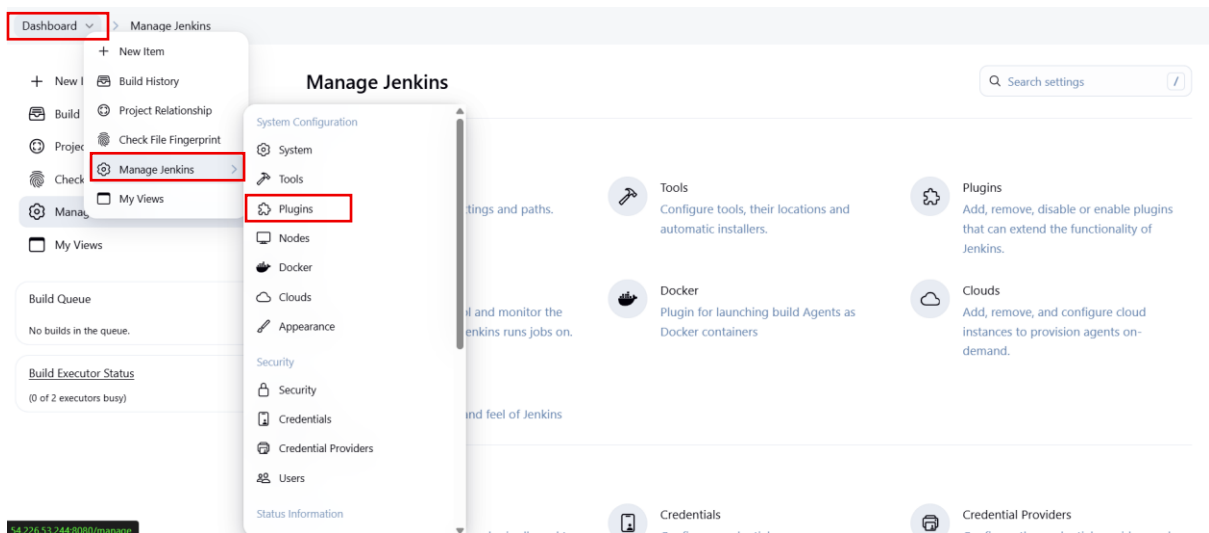
- Click on the Credentials under System Configuration



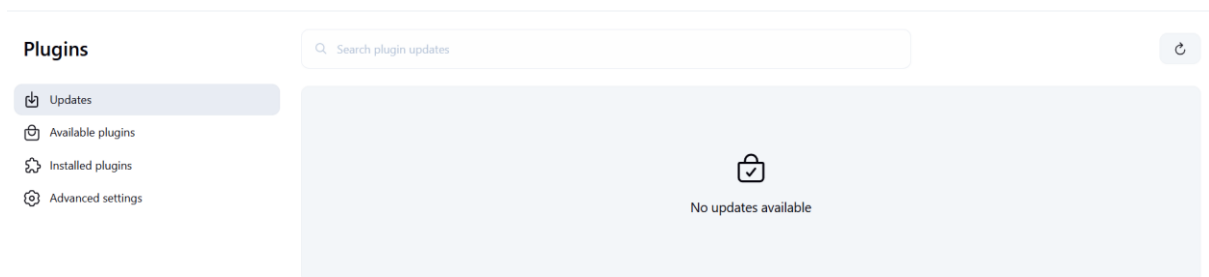
The screenshot shows the Jenkins 'System Configuration' page. The 'Plugins' option is highlighted with a red box. The page is divided into several sections: 'System' (Configure global settings and paths), 'Tools' (Configure tools, their locations and automatic installers), 'Nodes' (Add, remove, control and monitor the various nodes that Jenkins runs jobs on), 'Appearance' (Configure the look and feel of Jenkins), 'Docker' (Plugin for launching build Agents as Docker containers), 'Clouds' (Add, remove, and configure cloud instances to provision agents on-demand), 'Security' (Secure Jenkins; define who is allowed to access/use the system), 'Credentials' (Configure credentials), and 'Credential Providers' (Configure the credential providers and types).

OR

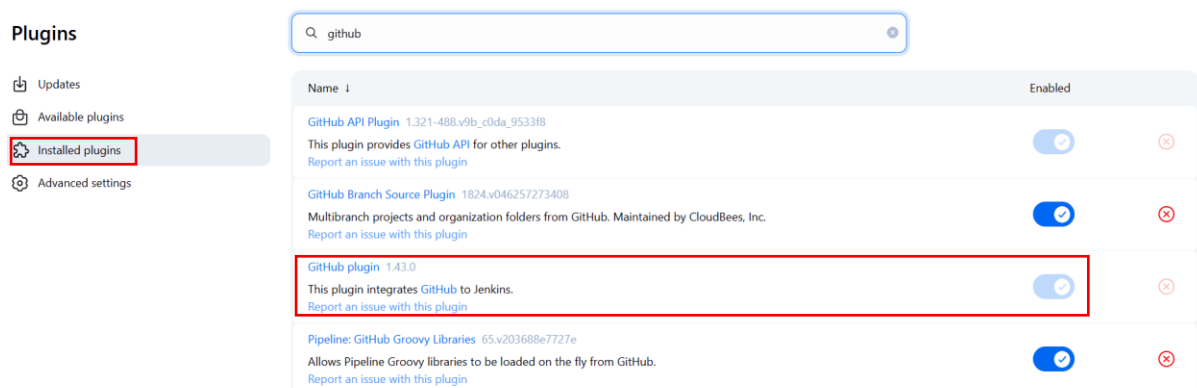
- You can directly visit to plugins through a dropdown list.
- Place your cursor on the Dashboard
- Then place your cursor on the Manage Jenkins
- Then click on the plugins



- You will be redirected to the plugin page.



- Lets check the installed plugins first
- There you will find many plugins but we will looking for github plugin
- We can search the github
- You can see github plugin is already downloaded here.



- Now we will download the docker plugin
- Click on the Available plugin.
- Search for docker
- Tick the docker plugin
- Click on install

Dashboard > Manage Jenkins > Plugins

Plugins

Search Docker

Install

Updates

Available plugins

Installed plugins

Advanced settings

Install	Name ↓	Released
	Docker 1274.vc0203fd2e74 Cloud Providers Cluster Management docker This plugin integrates Jenkins with Docker	4 mo 5 days ago

- You can see the download process.
- Click on Go back to the top page.

Dashboard > Manage Jenkins > Plugins

Plugins

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity

Docker

Pending

Loading plugin extensions

Pending

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running

Clone the GitHub repository using Freestyle Project

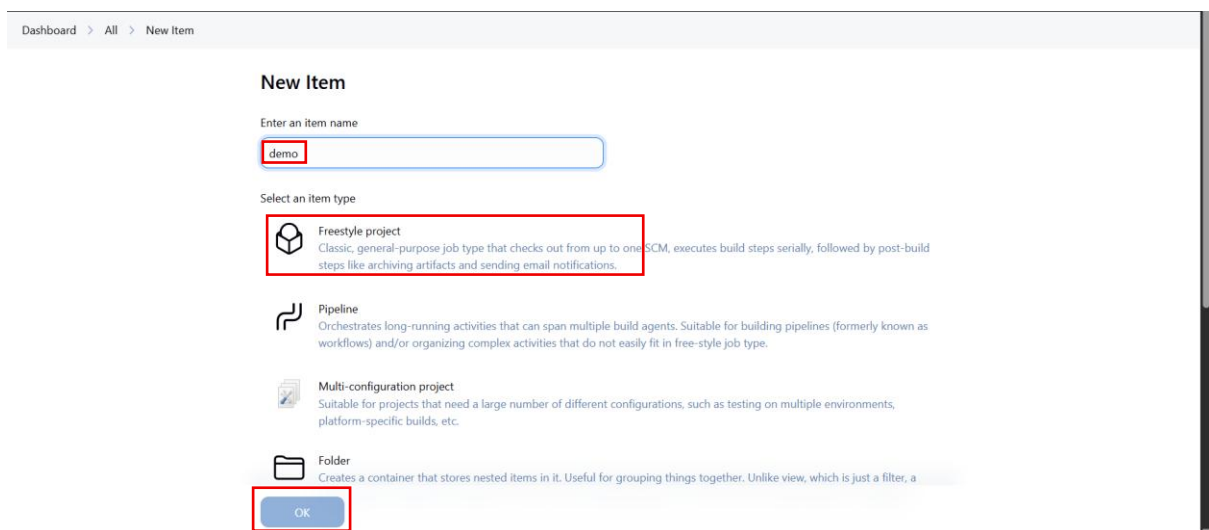
Summary of the Lab

This section shows how to clone a GitHub repository using Jenkins Freestyle Projects. After creating a new Freestyle job, users select **Git** under Source Code Management, enter the GitHub repo URL, and optionally choose credentials if the repo is private. Jenkins supports branches like main or master, which should be confirmed in GitHub. A build step is added to execute `cat README.md`, demonstrating how Jenkins accesses and uses repository files. After saving and running the job, output is verified in the Console Output. This setup confirms Jenkins' ability to pull and build code from Git repositories.

- Click on New Items.



- Give the name of the job
- Then select the freestyle project
- And click ok.



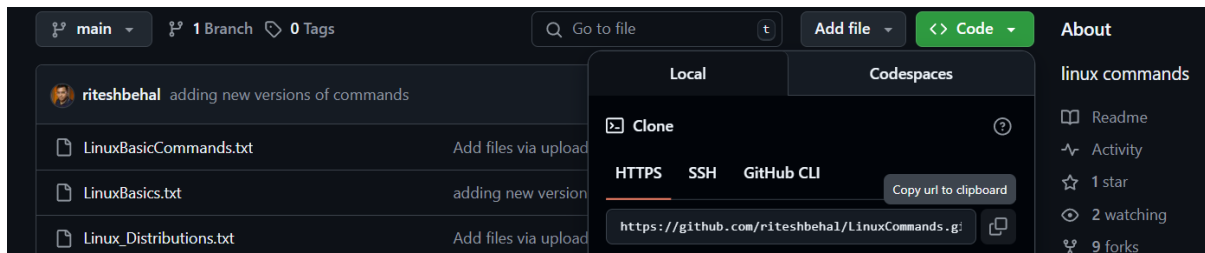
- Navigate Source Code Management
- Select the git option from there

Source Code Management

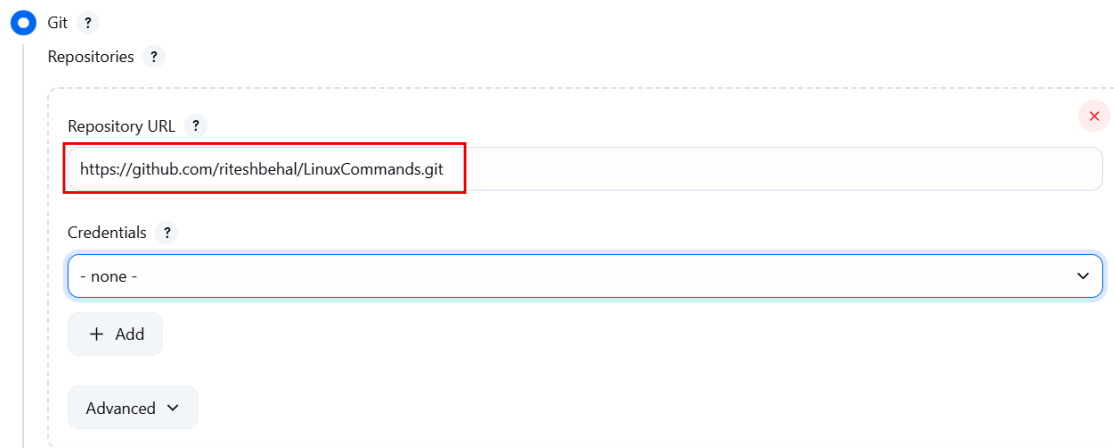
Connect and manage your code repository to automatically pull the latest code for your builds.

- ☐ None
- ☒ Git ?

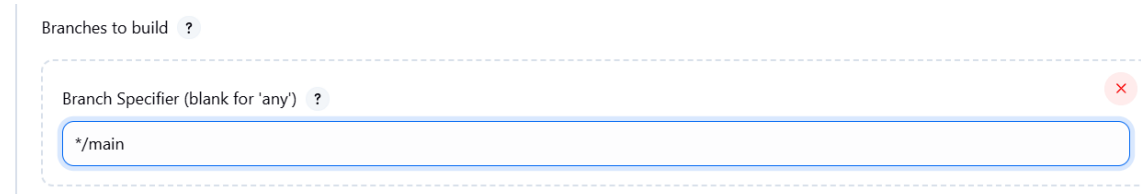
- After selecting git.
- Copy the URL from the github



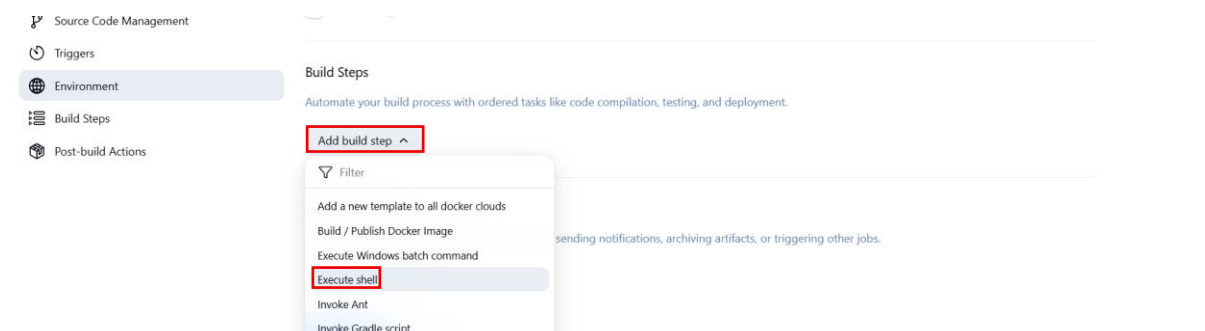
- Enter the Repository URL
- Note: If your repository is private, you have to select the credentials from the credential option.



- Check you github repository is master or main.



- Now come to Build Step section.
- Navigate to the Build Step section
- Click on the Add Build Step
- Select the 'Execute Shell'



- A shell will open.
- Type a simple command – “cat README.md”

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

≡ Execute shell ?

Command

See [the list of available environment variables](#)

```
cat README.md
```

Advanced ▾

- Click Save.
- After saving it will create a job successfully
- Now click on the Build now
- It will excute your job.
- When you will click on the ‘Build Now’ it sends a notification ‘build schedule’
- You can click on the #1 to see the results under build section.

Status

demo

Add description

</> Changes

Workspace

▶ Build Now

⚙️ Configure

🗑️ Delete Project

✎ Rename

Builds

No builds

Today
🟢 #1 3:35 PM

- Then you will be navigated to #1 build page.
- Then click on the Console Output.

Status

🟢 #1 (Jul 10, 2025, 3:35:05 PM)

Add description

Keep this build forever

</> Changes

📄 Console Output

✓ Edit Build Information

🗑️ Delete build '#1'

🕒 Timings

🕒 Started by user Aman Rathor

🕒 This run spent:

- 51 ms waiting;
- 97 ms build duration;
- 0.14 sec total from scheduled to completion.

</> No changes.

Started 4 min 1 sec ago

Took 97 ms

- You can see the status here.

</> Changes

Console Output

Edit Build Information

Delete build '#3'

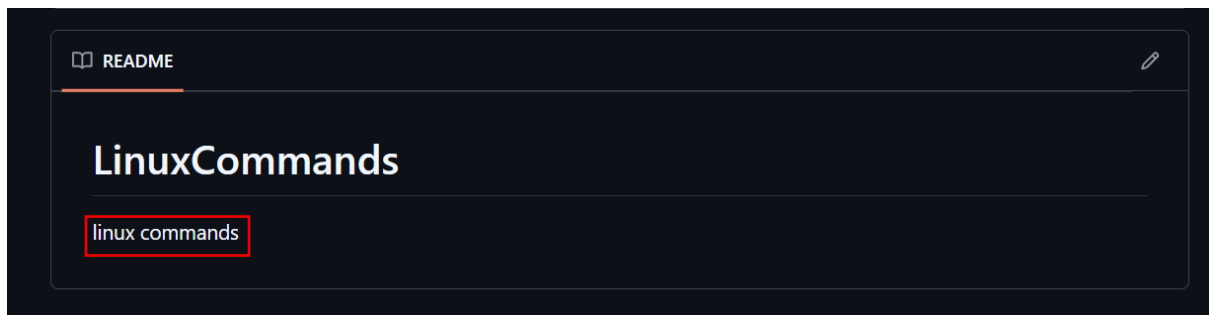
Timings

Git Build Data

← Previous Build

Started by user Aman Rathor
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/demo
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/riteshbehal/LinuxCommands.git
> git init /var/lib/jenkins/workspace/demo # timeout=10
Fetching upstream changes from https://github.com/riteshbehal/LinuxCommands.git
> git --version # timeout=10
> git --version # 'git version 2.47.1'
> git fetch --tags --force --progress -- https://github.com/riteshbehal/LinuxCommands.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/riteshbehal/LinuxCommands.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 87d498a05efd5b89ed27906480bd4a99daac9311 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 87d498a05efd5b89ed27906480bd4a99daac9311 # timeout=10
Commit message: "adding new versions of commands"
First time build. Skipping changelog.
[demo] \$ /bin/sh -xe /tmp/jenkins12650398234133204980.sh
+ cat README.md
LinuxCommands
linux commands
Finished: SUCCESS

- This is there in my README.md file



Clone the GitHub repository using Pipeline.

Summary of the Lab

Pipeline jobs offer advanced CI/CD capabilities. After creating a new Pipeline job, users choose **Pipeline script** as the definition. Jenkins' **Pipeline Syntax Generator** helps generate valid code snippets. First, Git repository information is added, then shell commands like `cat README.md` are scripted. These are combined into a single pipeline script. Once saved, the job is triggered with **Build Now**. Jenkins visually tracks the build process in stages, offering detailed logs and real-time progress. This method provides a scalable and automated way to manage code builds, tests, and deployments using code-as-configuration in Jenkins.

- Create a new item.
- Give the name of the job
- Select the pipeline
- Click OK.

Dashboard > All > New Item

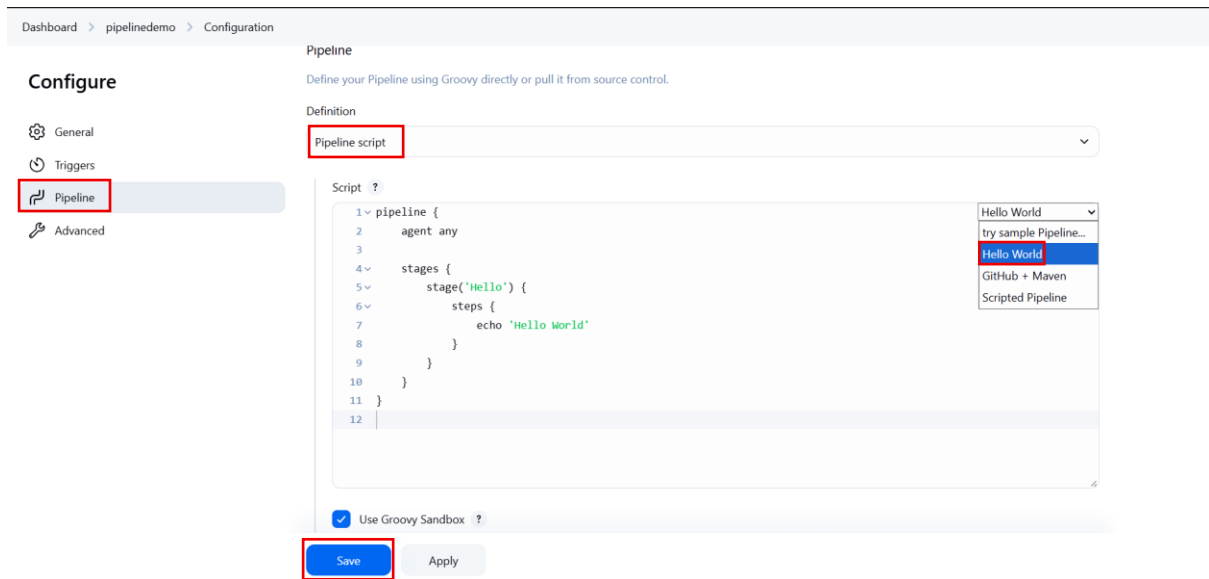
New Item

Enter an item name

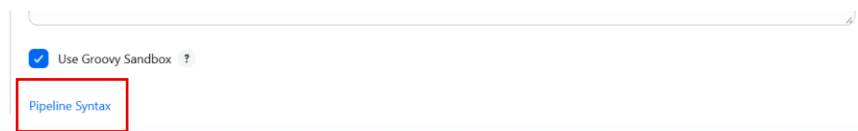
Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a

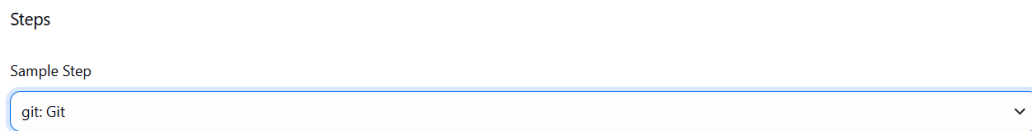
- Under the pipeline, in definition, select the option Pipeline script.
- Select the Hello World



- Now click on the pipeline syntax
- This pipeline syntax is used to generate the code.



- A new tab will open.
- Now in sample step, select git: Git



- Now enter the details

Sample Step

git: Git

git ?

Repository URL ?

`https://github.com/riteshbehal/LinuxCommands.git`

❗ Please enter Git repository.

Branch ?

`main`

Credentials ?

`- none -`

+ Add

☒ Include in polling? ?

☒ Include in changelog? ?

- Click generate pipeline script
- It will generate the code.
- Copy the code and paste in the pipeline.

Generate Pipeline Script

git branch: 'main', url: 'https://github.com/riteshbehal/LinuxCommands.git'



- Now in sampel step, select sh: Shell Script
- Copy this and paste it.

Steps

Sample Step

sh: Shell Script

sh

Shell Script ?

`cat README.md`

Advanced ▾

Generate Pipeline Script

sh 'cat README.md'

- This is a pipeline script.

Definition

Pipeline script

Script ?

```

1 pipeline {
2   agent any
3
4   stages {
5     stage('Cloning the Git Repo') {
6       steps {
7         git branch: 'main', url: 'https://github.com/riteshbehal/LinuxCommands.git'
8       }
9     }
10    stage('Display README File') {
11      steps {
12        sh 'cat README.md'
13      }
14    }
15  }

```

- Click Save.
- Now click on build now.
- Your #1 has created.
- Here, there is a option called stage to track the pipeline.
- Click on the stage to monitor the pipeline.

Dashboard > pipelinedemo >

Status

</> Changes

▷ Build Now

⚙️ Configure

🗑️ Delete Pipeline

📁 Stages

✎️ Rename

❓ Pipeline Syntax

pipelinedemo

✎️ Add description

Permalinks

Builds

*** 🔗

Today

🟢 #1 6:15 PM

54226321448080/job/pipelinedemo/configure

REST API Jenkins 2.504.3

Dashboard > pipelinedemo > Stages

Stages

▶ Build

⋮

July 10, 2025

🟢 #1

23:45 - 4.8s

Cloning the Git Repo

0.51s

○ - 🟢 - 🟢 - ○

- Now we can see the console output.



Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

```
[Pipeline] { (Cloning the Git Repo)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote git repository
Cloning repository https://github.com/riteshbhal/LinuxCommands.git
> git init /var/lib/jenkins/workspace/pipelinedemo # timeout=10
Fetching upstream changes from https://github.com/riteshbhal/LinuxCommands.git
> git --version # timeout=10
> git --version # 'git version 2.47.1'
> git fetch --tags --force --progress -- https://github.com/riteshbhal/LinuxCommands.git # timeout=10
> git config remote.origin.url https://github.com/riteshbhal/LinuxCommands.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 87d498a05efd5b89ed27906480bd4a99daac9311 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 87d498a05efd5b89ed27906480bd4a99daac9311 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b main 87d498a05efd5b89ed27906480bd4a99daac9311 # timeout=10
Commit message: "adding new versions of commands"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Display README file)
[Pipeline] sh
+ cat README.md
# LinuxCommands
Linux Commands
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```